SageAI / SageStrike

Project Development Lifecycle

Comprehensive Documentation

Generated: 2026-03-31
Version: 2.0 - Astronomical Edition

Table of Contents

1. Executive Summary
2. Project Overview
3. Phase 1: Planning & Requirements
4. Phase 2: Architecture & Design
5. Phase 3: Development
6. Phase 4: Testing & QA
7. Phase 5: Deployment
8. Phase 6: Operations & Maintenance
9. Workflow Diagrams
10. Risk Management
11. Resource Allocation
12. Timeline & Milestones
13. Appendices

1. Executive Summary

SageAI is an AI-powered enterprise security and OSINT platform designed for attack surface management, automated penetration testing, and advanced intelligence gathering. This document outlines the complete Software Development Lifecycle (SDLC) for both the original SageAI platform and the new SageStrike distributed attack engine. The project follows an enhanced Agile methodology with DevSecOps practices, incorporating continuous integration, automated security testing, and cloud-native deployment strategies. The development lifecycle spans 18 months with a team of 20+ engineers, security researchers, and DevOps specialists.

Metric	Value
Project Duration	18 Months
Team Size	20-25 Engineers
Architecture	Microservices (K8s)
Primary Language	Python 3.11+, Go 1.21+
Database	PostgreSQL + Redis + Kafka
Frontend	Next.js 15 + TypeScript
Deployment	Multi-cloud (AWS/GCP/Azure)

2. Project Overview

SageAI represents a paradigm shift in automated security testing, combining traditional vulnerability scanning with AI-powered analysis and distributed exploitation capabilities. The platform is built on a modern microservices architecture designed for horizontal scalability and cloud-native deployment.

2.1 Core Components

Component	Description
Intelligence Hub	9 OSINT modules for reconnaissance across public data sources
Multi-Agent AI	LLM-powered agents for reconnaissance, scanning, exploitation, and reporting
Distributed Scanner	Go-based high-performance port scanner with SYN stealth capability
Exploit Framework	Python-based exploitation engine with 10+ vulnerability classes
Risk Engine	CVSS-weighted scoring with AI-enhanced contextual analysis
Asset Management	Real-time attack surface inventory with relationship mapping

3. Phase 1: Planning & Requirements (Month 1-2)

3.1 Stakeholder Analysis

Primary stakeholders include enterprise security teams, bug bounty hunters, MSSPs (Managed Security Service Providers), and internal red teams. Requirements were gathered through 50+ interviews with CISOs, penetration testers, and security architects across Fortune 500 companies.

3.2 Requirements Categories

Category	Requirements	Priority
Functional	9 OSINT modules, Multi-agent pipeline, Real-time scanning	Critical
Performance	100k ports/sec, <5ms latency, 99.9% uptime	Critical
Security	Zero-trust architecture, Encryption at rest/transit	Critical
Scalability	100+ pod scaling, 1M+ targets/day	High
Compliance	GDPR, SOC2, ISO 27001 ready	High
Integration	REST API, WebSocket, Webhook support	Medium

4. Phase 2: Architecture & Design (Month 2-4)

The architecture follows microservices principles with event-driven communication. Each component is designed for independent deployment, scaling, and failure isolation. The system uses a polyglot persistence approach with specialized databases for different data patterns.

4.1 Technology Stack

Layer	Technology	Purpose
API Gateway	Kong/AWS API Gateway	Rate limiting, auth, routing
Frontend	Next.js 15 + Tailwind	React SSR, TypeScript
Backend API	FastAPI + Python 3.11	Async REST API
Scanning	Go 1.21	High-performance network I/O
Message Queue	Apache Kafka	Event streaming
Cache/Stream	Redis Cluster	Real-time data, sessions
Database	PostgreSQL + TimescaleDB	Relational + time-series
Graph DB	Neo4j	Relationship mapping
ML/AI	PyTorch + Transformers	Vulnerability detection
K8s Platform	EKS/GKE	Container orchestration

5. Phase 3: Development (Month 4-12)

Development follows trunk-based development with feature flags. The codebase is organized into microservices with clear API contracts defined in OpenAPI specifications. Each service has independent CI/CD pipelines with automated testing, security scanning, and deployment.

5.1 Development Workflow

1. Feature Request → Product Backlog Refinement
2. Sprint Planning → Story Point Estimation
3. Feature Branch Creation → TDD Development
4. Code Review (2+ approvals required)
5. Automated Testing (Unit, Integration, E2E)
6. Security Scanning (SAST, DAST, Dependency)
7. Merge to Main → Staged Deployment
8. Canary Release → Production Rollout

5.2 Service Architecture

Service	Language	Responsibility	Scale
api-gateway	Go	Request routing, auth	3 pods
web-frontend	TypeScript	Next.js SSR app	5 pods
scan-orchestrator	Python	Scan coordination	3 pods
scan-worker	Go	Port scanning	10-100 pods
exploit-engine	Python	Vulnerability exploitation	5-50 pods
intelligence-hub	Python	OSINT data collection	5 pods
ai-agent	Python	LLM analysis	3 pods
event-processor	Python	Kafka consumers	5 pods
report-generator	Python	PDF/report creation	2 pods

6. Phase 4: Testing & QA (Continuous)

6.1 Testing Pyramid

Level	Coverage Target	Tools	Environment
Unit Tests	80% code coverage	pytest, Go test	Local/CI
Integration Tests	API contracts	pytest-asyncio, TestContainers	CI
Contract Tests	All service APIs	Pact	CI
E2E Tests	Critical user paths	Playwright, Cypress	Staging
Security Tests	OWASP Top 10	Bandit, Semgrep, ZAP	CI/Staging
Performance Tests	100k req/sec	k6, Locust	Staging/Prod

6.2 Quality Gates

- All unit tests passing (>80% coverage)
- No critical/high security vulnerabilities (SAST/DAST)
- Performance benchmarks within 10% of baseline
- Code review approved by 2+ senior engineers
- Documentation updated (API specs, changelogs)
- Feature flags configured for rollback

7. Phase 5: Deployment (Month 12-14)

7.1 Deployment Strategy

The platform uses a GitOps workflow with ArgoCD for Kubernetes deployment. Each service has independent deployment pipelines with automated canary releases. The deployment environment progression: Local → CI → Development → Staging → Production.

7.2 Infrastructure as Code

Component	Tool	Purpose
K8s Manifests	Kustomize + Helm	Service deployment
Infrastructure	Terraform	AWS/GCP/Azure resources
Networking	Cilium	Service mesh, policies
Secrets	Vault	Secret management
Monitoring	Prometheus Operator	Metrics collection
Logging	Loki + Fluentd	Log aggregation
Tracing	Jaeger	Distributed tracing

8. Phase 6: Operations & Maintenance (Ongoing)

8.1 Monitoring & Observability

Category	Metrics	Alert Threshold
Performance	P95 latency, throughput	Latency > 100ms
Reliability	Error rate, uptime	Errors > 0.1%
Security	Failed auth, anomalies	Any critical alert
Business	Active scans, findings	Queue depth > 1000
Infrastructure	CPU, memory, disk	Resource > 80%

8.2 Incident Response

P0 (Critical): Service down, security breach - Response: 15 min, Resolution: 2 hours P1 (High): Major feature degraded - Response: 1 hour, Resolution: 4 hours P2 (Medium): Partial impact - Response: 4 hours, Resolution: 24 hours P3 (Low): Minor issues - Response: 24 hours, Resolution: 72 hours

9. Workflow Diagrams

9.1 CI/CD Pipeline Workflow

1. Developer pushes code to feature branch
2. GitHub Actions triggers: Lint → Unit Tests → Build
3. Security scanning: SAST (Bandit/Semgrep), Dependency check
4. Docker image built and pushed to ECR with SHA tag
5. Integration tests run against ephemeral environment
6. PR reviewed and merged to main branch
7. ArgoCD detects change and updates staging
8. Canary deployment: 10% → 50% → 100% traffic
9. Automated rollback if error rate > threshold
10. Production deployment with feature flags

9.2 Data Flow Architecture

User Request → Kong Gateway → Auth Service → API Gateway → Kafka Event Bus → Service Consumers → PostgreSQL/Redis → WebSocket → User Dashboard Scanning: Coordinator → Kafka → Scan Workers → Results → Exploit Engine → Findings DB → AI Analysis → Report Generator

9.3 Multi-Agent AI Workflow

ReconAgent: Subdomain enumeration → DNS resolution → Shodan enrichment ↓ ScannerAgent: Tool selection → Nmap/SSL/DNS execution → Vulnerability triage ↓ ExploitAgent: Safe exploitation → Evidence collection → Risk assessment ↓ ReportAgent: Executive summary → Risk scoring → Attack path analysis ↓ Delivery: PDF report → WebSocket notification → Dashboard update

10. Risk Management

10.1 Technical Risks

Risk	Probability	Impact	Mitigation
K8s complexity	High	High	Training, runbooks, managed EKS
Performance at scale	Medium	High	Load testing, caching, rate limiting
Security vulnerabilities	Medium	Critical	Penetration testing, bug bounty
AI model failures	Medium	Medium	Fallback rules, human oversight
Dependency vulnerabilities	High	High	Automated scanning, SLSA compliance
Data breaches	Low	Critical	Encryption, zero-trust, audit logging

11. Resource Allocation

11.1 Team Structure

Role	Count	Responsibility
Engineering Manager	1	Team coordination, roadmap
Tech Lead	1	Architecture decisions, mentoring
Senior Backend Engineers	4	Core services, APIs
Backend Engineers	4	Feature development
Go Developers	2	Scanner, performance-critical
Frontend Engineers	3	Next.js, React components
DevOps Engineers	2	K8s, CI/CD, infrastructure
Security Engineers	2	Exploits, hardening, audits
ML Engineers	2	AI agents, model training
QA Engineers	2	Testing, automation
Product Manager	1	Requirements, prioritization
UI/UX Designer	1	Design system, prototypes

11.2 Infrastructure Costs (Monthly)

Component	Specification	Monthly Cost
EKS Cluster	3 nodes x c5.2xlarge	$1,200
Worker Nodes	20-100 x c5.4xlarge (spot)	$3,000-15,000
Kafka	3 x r5.xlarge	$1,200
PostgreSQL	RDS db.r5.2xlarge	$800
Redis	ElastiCache r5.xlarge	$400
Load Balancers	ALB x 3	$300
Storage	5TB EBS + S3	$500
Networking	NAT, data transfer	$400
Monitoring	CloudWatch, Datadog	$600
Total		**$8,400-22,400**

12. Timeline & Milestones

12.1 Release Schedule

Milestone	Date	Deliverables	Status
MVP	Month 3	Core API, basic scanning	Completed
Alpha	Month 6	9 OSINT modules, auth	Completed
Beta	Month 9	Multi-agent AI, reports	Completed
v1.0	Month 12	Production launch	In Progress
v1.5	Month 15	SageStrike integration	In Progress
v2.0	Month 18	Advanced exploits, ML	Planned
v2.5	Month 21	Dark web, custom ASIC	Planned
v3.0	Month 24	Autonomous pentesting	Planned

13. Appendices

Appendix A: Definition of Done

A feature is considered "Done" when: • Code is written and reviewed by 2+ engineers • Unit tests achieve >80% coverage • Integration tests pass in CI • Documentation is updated (API specs, user guides) • Security scan passes with no critical/high issues • Performance benchmarks meet targets • Feature flags are configured • Monitoring and alerting are in place • Runbook is updated for operational procedures

Appendix B: Security Requirements

All code must adhere to: • OWASP ASVS Level 2 compliance • All secrets in Vault (never in code) • Input validation on all API endpoints • SQL injection prevention (parameterized queries) • XSS prevention (output encoding) • CSRF protection for state-changing operations • Rate limiting on all public endpoints • Encryption at rest (AES-256) and in transit (TLS 1.3) • Audit logging for security events

Appendix C: Performance Targets

Scanner Performance: • Port scanning: 100,000 ports/second per pod • API response time: P95 < 100ms • WebSocket latency: < 50ms • Database queries: P95 < 10ms • Kafka throughput: 50,000 events/second • System availability: 99.9% uptime • Concurrent scans: 10,000+ simultaneous • Report generation: < 30 seconds for 1000 findings